Local Memory-System Benchmark

Design, question taxonomy, and per-category performance analysis

Companion Intelligence — CI-Server

June 2026

Private & Confidential — Property of Lifescope Inc. Do not distribute.

Abstract

This document specifies a **fully local** benchmark for agent-memory systems — the storage and retrieval layer that lets an assistant remember facts about its user across a conversation — and analyses *why* different architectures succeed or fail on different kinds of question. Seven systems are evaluated on a single synthetic user under a uniform, model-controlled protocol with no cloud calls. The instrument is a **657-message single-user corpus** (257 canonical facts buried under 400 distractors) probed by **460 questions across eight capability classes**. The document's purpose is twofold: (1) to define the **taxonomy** — what each question class tests, why it matters, how it is scored, with worked examples — so that a score is interpretable as a specific memory capability; and (2) to explain the **per-category results mechanistically** — why graph structure wins complete-set and multi-hop queries, why raw-text retrieval wins single-hop and temporal queries, and why their **union** (CI-Server's hybrid configuration, **91.1%**, field-best) dominates by inheriting both. A recurring theme: headline accuracy hides the real story, which lives in the per-category profile.

1 Purpose and scope

An agent-memory system ingests a stream of user statements and later answers questions about them. Two systems with identical headline accuracy can have completely different competence profiles — one reliably returns complete sets but fumbles single facts, another the reverse. This benchmark is built to **expose those profiles**, not just rank systems. Every design choice serves that goal: a fixed, shared answerer so we measure the *memory layer* and not the writer; a taxonomy whose classes map to distinct retrieval mechanisms; and two scored dimensions (was the fact *retrieved*; was it *answered*) whose gap is itself diagnostic.

The constraint throughout is **local-only**: no cloud APIs, no hosted SaaS. Every model runs on-device. This excludes cloud-only providers (Honcho, RetainDB, Supermemory, Memori) and forces self-hosted configurations of the rest — e.g. Mem0 as its open-source library over a local vector store, not its hosted service.

2 Benchmark design

2.1 One user is the only real use case

Each system under test is built for a *single deployment serving a single user*; none has a first-class notion of separate accounts inside one store. This is not incidental — it is the production use case for a personal assistant. An early design modelled three *independent* users in one store; it

was abandoned because it measured identity-handling artifacts that never arise in deployment (a consolidation store would bind one user's first-person facts to another user's named entity; a graph store would mis-anchor them to its canonical "self" node). The definitive corpus therefore models **one user, "Max," plus a rich network of people he knows**: two friends (Priya, Jordan) and their families and colleagues are folded in as third-party facts that Max states. This keeps the benchmark realistic *and* naturally generates deep relational and multi-hop structure (Max → friend → their relative → attribute).

2.2 Two-phase, model-controlled protocol

Every system runs in its own container exposing a uniform HTTP contract (`reset`, `ingest`, `flush`, `retrieve`, `answer`); the orchestrator speaks only this contract, so each system is a black box behind the same interface.

- **Phase A — ingest & extraction.** Every system ingests the identical corpus using the same extractor LLM (`gemma-4-31b-it-abliterated`) and the same embedding model (`nomic-embed-text`).[1]
- **Phase B — answer & judge.** A *single, constant* retrieval-augmented answerer and LLM judge (`ornstein-3.6-27b`) serves *all* systems. Given a question, each system retrieves its own context; that context is handed to the *same* answerer; the answer is graded by the *same* judge. We thus compare memory layers, holding the reader/writer constant.

2.3 Two scored dimensions

Each question yields two measurements:

- **Answer** — did the constant answerer, given the system's retrieved context, produce the correct final answer? (the headline metric.)
- **Storage** — was the gold fact actually *present in the raw retrieved context*, regardless of whether the answer was right?

The **gap between them is diagnostic**. High storage with lower answer means the fact was retrieved but buried in noise or ambiguity (a *precision* problem). Low storage means the fact was never surfaced (a *recall* problem). We use this gap repeatedly in §6.

2.4 Corpus

257 hand-authored canonical facts — people, places, employers, multi-hop chains, and facts that change over time — are buried under **400 distractor messages** so that a fixed top-k retriever cannot win by returning everything. Total **657 messages**; a top-10 retriever sees ∼1% of the corpus per query. Every gold answer is grounded in exactly one canonical fact, so a miss is always a retrieval/answer failure, never a missing fact.

3 Question taxonomy

460 questions across **eight capability classes**. Each class isolates a distinct memory operation; the parenthetical is the question count. Because the two friends are modelled as *people Max knows*, their own attributes are **relational** (one hop from Max) and their networks are **multi-hop**.

[1] Two flagged exceptions: Hindsight uses its built-in `bge-small` embedder; Holographic uses *no* LLM and *no* embeddings (keyword + algebraic binding). Both are noted at the relevant rows.

3.1 `flat-recall` (34 questions)

Tests: retrieval of a single atomic fact about the user, stated once and never changed. **Why:** the simplest memory operation and the floor for any system — if you cannot return a fact you were told verbatim, nothing else matters. **Scoring:** LLM judge (semantic match to the gold answer). **Examples:**

- *"What is Max's profession?"* → robotics engineer
- *"What instrument does Max play?"* → violin
- *"What is the name of Max's cat?"* → Biryani

3.2 `relational` (90 questions)

Tests: a fact reached by *one hop* from the user — an attribute of a person, place, or thing Max is directly connected to. **Why:** real assistant queries are rarely about the user in isolation ("what does my sister do," "where do my parents live"); one-hop traversal is the common case. The class is large (90) because it is where terse, structured stores tend to lose the *relation* that disambiguates an answer. **Scoring:** LLM judge. **Examples:**

- *"What is the name of Max's sister?"* → Anita
- *"Where do Max's parents live?"* → Portland
- *"What does Max's friend Priya do for work?"* → perception engineer

3.3 `multi-hop` (118 questions)

Tests: a fact reached by *two or more hops* (Max → A → B → attribute). The largest class, and the one structure is supposed to own. **Why:** answering "what does my best friend's wife do" requires chaining relationships, not matching a sentence; it separates systems that *model* connections from those that merely store text. **Scoring:** LLM judge. **Examples:**

- *"What does Max's brother-in-law do for work?"* → cardiologist
- *"Where does Max's best friend work?"* → Stripe
- *"What does Priya's husband do?"* (Max → Priya → husband → occupation)

3.4 `temporal-update` (90 questions)

Tests: facts that *changed* over the corpus — the system must return the *current* value and not the superseded one (and sometimes recall the prior value on request). **Why:** memory that never updates is worse than useless; a system that answers "Portland" after the user moved to Denver is actively wrong. Probes supersession and recency handling. **Scoring:** LLM judge (current value required; stale value marked wrong). **Examples:**

- *"What city does Max currently live in?"* → Denver (he previously lived in Portland)
- *"What company does Max currently work for?"* → Cobalt Robotics
- *"What city did Max live in before his current one?"* → Portland

3.5 `enumeration` (34 questions)

Tests: returning a *complete typed set* ("list all X"), not just one member. **Why:** completeness is a different skill from point lookup — a store can reliably return *a* family member yet never assemble *all* of them. This is the structural store's natural strength (typed edges enumerate cleanly) and the keyword store's natural weakness. **Scoring: deterministic set recall** — the fraction of gold members surfaced, no LLM judge, nothing to spin. **Examples:**

- *"List all of Max's family members."* → {Anita, Deepak, Sunita, Raj, Mira, Felix, ...}
- *"Name every company Max has worked for."* → {Cobalt Robotics, Northwind Logistics}
- *"List the musical instruments played by people in Max's life."* → {violin, viola, harp}

3.6 `aggregation` (23 questions)

Tests: a count or filter computed *over* the stored facts ("how many..."). **Why:** requires not just retrieval but correct counting of the retrieved set — and punishes stores that surface a polluted block, since a stray entity inflates the count. **Scoring:** LLM judge against the gold count. **Examples:**

- *"How many companies has Max worked for?"* → two (Cobalt Robotics, Northwind Logistics)
- *"How many cities has Max lived in?"* → two (Portland, Denver)
- *"How many children does Max's sister Anita have?"* → two (Mira, Felix)

3.7 `absence` (48 questions)

Tests: correctly answering **"no"** to a plausible-but-false premise — each baited with a tempting distractor about *someone else*. **Why:** a memory system that hallucinates a confident "yes" to something it was never told is dangerous; this class measures resistance to that failure. The bait ensures a system cannot pass by keyword-matching a near-miss. **Scoring:** existence-aware judge; a confident "yes" to a non-existent thing is a hallucination (wrong). **Examples:**

- *"Does Max have a brother?"* → No (one sibling, his sister Anita; his *brother-in-law* Raj is the bait)
- *"Does Max play the piano?"* → No (he plays violin; his grandmother was the pianist — bait)
- *"Does Max own a dog?"* → No (his only pet is a cat, Biryani)

3.8 `presence-control` (23 questions)

Tests: genuine "yes" questions, structurally identical to the absence class. **Why:** a *control* — without it, a system could game the absence suite by answering "no" to everything. Strong absence *and* strong presence-control together demonstrate genuine discrimination. **Scoring:** existence-aware judge. **Examples:**

- *"Does Max have a sister?"* → Yes (Anita)
- *"Does Max own a cat?"* → Yes (Biryani)
- *"Does Max play a musical instrument?"* → Yes (the violin)

4 Systems under test

Seven systems span four retrieval philosophies. Understanding the architecture predicts the per-category profile (§6).

Naive RAG (vector)
Chunk-level vector store: raw messages embedded with `nomic`, top-k cosine retrieval. No structure — the control for "just retrieve the text."

Mem0 (consolidating vector)
Vector store with *LLM-driven consolidation*: at write time it merges and rewrites related statements into dense, deduplicated memory records.

CI-Server (graph)
Extracts typed entities and relationships into a property graph; recall is an entity search plus a one-hop expansion over *serialized nodes* (name + description). Structure-first.

CI-Server (hybrid)
The graph above *unioned* with a raw-message vector channel (§7) — structure for traversal, raw text for fidelity.

OpenViking (context DB)
Volcengine context database; VLM-based extraction into a structured context store.

TencentDB Agent Memory
SQLite + `sqlite-vec`; per-message extraction with vector recall.

Holographic (keyword + HRR)
SQLite FTS5 keyword search plus Holographic Reduced Representations — *no* LLM and *no* embeddings. The minimal-cost baseline.

Hindsight (retain/recall)
A retain/recall service with `bge-small` embeddings; serialized recall (its client errors under concurrency).

5 Results

Table 1: Per-category answer score (460 questions). Headline is answer accuracy with 95% Wilson CI; bold marks the per-column leader. CI-Server's two configurations are highlighted — *graph* (vanilla) and *hybrid* (graph ∪ raw-message vector search).

System	Answer (95% CI) /460	flat /34	rel /90	m-hop /118	temporal /90	enum /34	aggr /23	abs /48	pres /23
CI-Server (hybrid)	**91.1%** ±2.6	**32**	85	102	**86**	**30.0**	17	**46**	21
mem0	86.9% ±3.1	23	83	**109**	78	24.9	17	43	**22**
openviking	80.2% ±3.6	26	69	106	61	24.9	17	43	**22**
naive	78.3% ±3.8	26	**86**	80	64	23.4	**18**	42	21
tencentdb	76.2% ±3.9	26	79	77	64	24.7	17	44	19
CI-Server (graph)	75.9% ±3.9	25	60	96	66	27.0	14	40	21
holographic	69.9% ±4.2	25	70	86	56	13.6	8	43	20
hindsight	22.3% ±3.8	1	12	15	14	6.8	5	42	7

Reading the numbers. At $N = 460$ the headline CI is ±3–4 pp; *per-category* intervals are far wider — relational/temporal ±8 pp, absence ±11 pp, flat/enumeration ±13 pp, aggregation/presence ±16 pp. Treat single-category gaps under those margins as noise, large ones as real. The headline therefore tells you little; the *shape* of each row — which categories it wins and loses — is the system's fingerprint, analysed next.

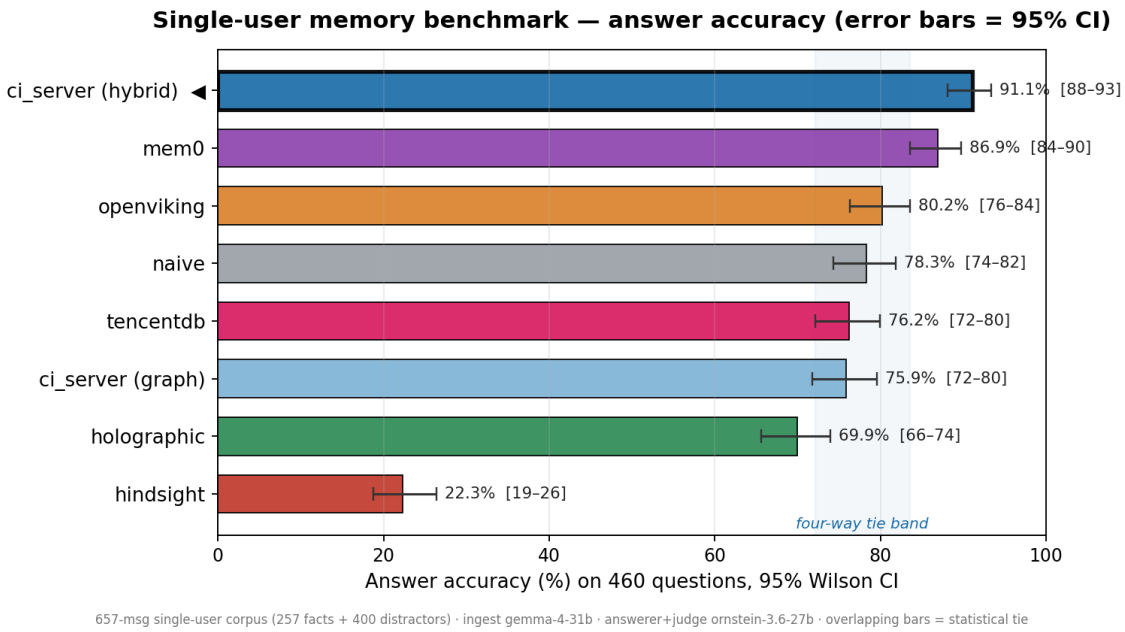


Figure 1: Answer accuracy with 95% Wilson confidence intervals. Overlapping bars are statistical ties. The hybrid configuration leads; the vanilla graph sits in a four-way tie at 76–80%.

6 Per-category performance analysis

The headline ranks systems; the per-category profile explains them. Each architecture has a signature.

`flat-recall` — **raw text wins, distillation loses.** The atomic fact ("violin") is present verbatim in one source sentence, so any store that keeps the sentence returns it (Naive/OpenViking/TencentDB 26; hybrid **32**). Mem0 is *lowest* of the serious systems (23): its consolidation rewrites statements and occasionally drops an atomic detail in favour of a summary. The graph (25) usually has the fact on the node but sometimes only as a terse field that the answerer reads past.

`relational` — **the graph's weak spot, the raw sentence's strength.** Naive leads (**86**) because "Max's sister Anita" lives in a single chunk that contains both the *relation* and the *answer*. The vanilla graph is the field's worst (60): distilling "Anita" into a `sister` edge and a terse node strips the surrounding context, so the answerer retrieves the right entity but cannot tell which relation it satisfies, or answers a competing entity ("Max has two managers"). This single weakness — storage (366.7) far exceeding answer (349.0) — is the clearest precision failure in the study, and the one the hybrid most directly fixes (60 → 85).

`multi-hop` — **structure and consolidation beat flat retrieval.** Here the ordering inverts: Mem0 leads (**109**) because its dense records *bundle* a chain of related facts into one retrievable unit; OpenViking (106) and the graph configurations (96/102) traverse explicitly; Naive is the worst of the capable systems (80) because flat chunks cannot chain — a two-hop answer requires two different chunks to co-occur. This is the category that justifies structure at all.

`temporal-update` — **the hybrid's largest margin.** The hybrid (**86**) dominates because both modalities help: the graph carries supersession (current vs. prior edges) and the raw channel carries the explicit sentences ("moved to Denver," "before that, Portland"). Single-modality stores trail (Mem0 78, graph 66, Naive 64): consolidation can overwrite the prior value, flat retrieval can surface the stale chunk with equal rank to the fresh one.

`enumeration` — **structure assembles sets; keyword collapses.** The typed graph enumerates cleanly (graph 27, hybrid **30**): a "list all family" query walks the typed edges and returns the complete set. Flat and consolidating stores miss members (Naive 23.4, Mem0 24.9 — whichever chunks fall outside top-k are simply absent). Holographic *collapses* (13.6): keyword matching cannot assemble a typed set it was never asked for by name. This is deterministic set recall — the one unspinnable column, and the graph's signature strength, retained by the hybrid.

`aggregation` — **uniformly hard; counting prose is brittle.** No system clears 18/23. The task asks the answerer to *count* a retrieved set, and an LLM tallying prose miscounts whenever the context is polluted — the graph is worst (14) because a one-hop expansion can pull a neighbour's relative into the block ("5 employers," counting a friend's sister). The robust fix is to count *typed edges on the graph* rather than ask an LLM to tally text — correct by construction, though this benchmark's $N = 23$ cannot resolve the delta.

`absence` / `presence-control` — **discrimination, not retrieval.** These move together and cluster tightly (most systems 40–46 / 19–22) because they test the *answerer's* willingness to say no, given context, more than retrieval depth. The hybrid edges absence (**46**): richer context lets the answerer confirm a thing is genuinely absent rather than guess. Presence-control sits near ceiling for everyone — a useful confirmation that no system games the absence suite by reflexively answering "no."

Summary of signatures. *Vector/raw-text* systems own single-hop, flat, and temporal (the source sentence preserves context); *structural* systems own enumeration and help multi-hop (typed edges assemble and chain); *consolidation* (Mem0) is the strongest single modality on multi-hop; *keyword* (Holographic) is cheap but collapses where typed structure is required. No single modality is best everywhere — which is exactly why the union wins.

7 The hybrid retrieval channel

The per-category analysis points to one conclusion: the graph and the raw text are *complementary*, and the gap between CI-Server's vanilla graph (76%) and the leaders was the text it discarded. An extraction graph embeds only the distilled *nodes* (name + description) and drops the source sentence; the detail that disambiguates a relational or temporal answer lives in that sentence.

Mechanism. The hybrid adds a second retrieval channel: a vector search over the *raw messages*, unioned with the graph traversal. Every recall returns graph nodes *plus* one-hop expansion *plus* the nearest source sentences. Structure provides enumeration and multi-hop reach; the raw text provides single-hop and temporal fidelity. The merge is the *plain union with no cap or re-ranking*: on a soft-spot-weighted sweep the uncapped union beat reciprocal-rank fusion and relevance re-ranking by $\sim$6 pp, because capping drops answer-bearing lines and the reasoning answerer already tolerates the larger context.

Result. Implemented natively (CI-Server embedding and searching its own messages through the existing `nomic` pipeline, no second store): **91.1%**, field-best. The per-category recovery is exactly what the analysis predicts — relational 60→85, temporal 66→86, flat 25→32 (the raw-text categories) — *while retaining* enumeration (27→30) and multi-hop (96→102) (the structural categories). The hybrid is not a better graph; it is the graph plus the modality the graph was missing.

8 Cost and operational characteristics

Memory-layer work only (the shared answer phase is excluded); ingest scaled to the 657-message corpus.

System	Ingest (657 msgs)	Recall latency/query	Concurrency-safe
holographic	∼**7 s** (no LLM/embeddings)	∼**5 ms**	yes
naive	∼20 s	∼60 ms	yes
mem0	∼20 m	∼120 ms	yes
CI-Server	∼16 m	∼80 ms	yes
openviking	∼70 m (VLM extraction)	∼30 ms	yes
tencentdb	∼100 m (per-message extraction)[2]	∼25 ms	yes
hindsight	∼3 h (cannot truly parallelize)	∼1160 ms	no[3]

The hybrid channel adds negligible cost: messages are embedded with the same `nomic` model already used at ingest, and the extra recall step is one vector query (∼20 ms). Structure-heavy extraction (OpenViking, TencentDB) dominates Phase-A wall-clock; the keyword baseline (Holographic) is two orders of magnitude cheaper but pays for it in enumeration.

9 Conclusions

- **The per-category profile is the result; the headline is a summary of it.** Each architecture has a signature — raw text for single-hop/flat/temporal, structure for enumeration/multi-hop, consolidation for multi-hop — and these are mechanistic, not incidental.
- **No single modality is best everywhere, so the union wins.** CI-Server (hybrid) leads at 91.1% by combining graph traversal with raw-text vector retrieval, recovering the graph's single-hop precision while keeping its complete-set strength. The transferable lesson: *an extraction pipeline should not discard its source text* — re-introducing it was worth +15 pp.
- **Mem0 is the strongest single-modality system** (86.9%, best multi-hop) but identity-fragile on multi-entity corpora; raw retrieval is robust.
- **Interpret per-category numbers against the noise floor** (±8 to ±16 pp); and remember these systems are single-user by design, so a multi-user benchmark measures artifacts, not capability.

A Methodological notes and threats to validity

Distance metric.
`nomic` embeddings are unnormalised ($\|v\| \approx 22$), so vector recall must use cosine (`<=>`) not L2 (`<->`); an L2 threshold silently returns nothing. All vector paths (Naive, the hybrid channel, the graph's semantic mode) use cosine.

Judge robustness.
`ornstein` is a reasoning model that spends tokens before emitting the schema-constrained verdict; it is given a generous token budget and retries transient failures, with a loud `JUDGE_UNSCORED` flag rather than a silent wrong. All reported numbers use the hardened judge (0 judge failures).

Resolution limit.
Per-category effect sizes below the noise floor (± 8 to ± 16 pp) are not measurable at this N. Small read-side recall tweaks to CI-Server fell below it; only the hybrid channel — a large, additive lever — cleared it. A focused single-class micro-suite (200+ questions) is the instrument for finer effects.

Extraction non-determinism.
`gemma` extraction varies run-to-run, so any change requiring a re-ingest carries an extraction-variance confound; same-store read-side comparisons (e.g. disabling the graph expansion: -30) are the clean ones.

Single-user validation.
The abandoned multi-persona design is retained only as evidence for the single-user design choice (§2.1): consolidation stores bound a second persona's first-person facts to a prominent other entity; the graph mis-anchored them to its canonical self node.

Reproduce

All artifacts live in `scripts/benchmark/v2/`: `orchestrator.py` (harness + judge), `docker-compose.yml` (one container per system), personas in `../personas/` (`max_expanded_large.json` is the definitive corpus), per-suite reports (`report_expanded_*.md`), and charts in `charts/`. The hybrid line is built by `hybrid_score.py`, `hybrid_tune.py`, and `embed_messages.py`; native code lives in `backend/apps/api/src/domains/` (`search-next/SearchService`, `agent/RecallService`, `agent/AgentMemoryService`).